

Exercises 4

Exercise 1 (First-Order Formulas). Among the following formulas of first order, classical logic, four are theorems and two are not. Prove two of them using usual pen-and-paper (but still reasonably formal) mathematical proofs, and two using formal resolution proofs. For the two that are not theorems, produce models (base set and interpretation) where they do not hold.

1. $(\exists y. \forall x. P(x, y)) \rightarrow \forall x. \exists y. P(x, y)$
2. $\exists x. \forall y. (Q(x) \rightarrow Q(y))$
3. $(\neg \forall x. \exists y. P(x, f(y))) \vee \exists y. \forall x. P(f(y), x)$
4. $(\forall x, y, z. P(x, y) \wedge P(y, z) \rightarrow P(x, z)) \wedge (\forall x, y. P(x, y) \vee P(y, x)) \rightarrow \forall x. P(x, x)$
5. $\exists x. P(x) \wedge P(f(x)) \rightarrow \forall y. P(y) \vee P(f(y))$
6. $\forall x. \forall y. P(x, y) \rightarrow \forall x. \forall y. P(y, x)$

Solution:

1. **True.** Suppose $\exists y. \forall x. P(x, y)$. Let y_0 be such that $\forall x. P(x, y_0)$. Now, we have to prove $\forall x. \exists y. P(x, y)$. Choose an arbitrary x_0 . We have $P(x_0, y_0)$, and hence $\exists y. P(x_0, y)$. Since x_0 was arbitrary, it follows that $\forall x. \exists y. P(x, y)$.
2. **True.** The formula is equivalent to $\exists x. \neg Q(x) \vee (\forall y. Q(y))$, and then to $(\exists x. \neg Q(x)) \vee (\forall y. Q(y))$. Finally, this reduces to $(\neg \forall x. Q(x)) \vee (\forall y. Q(y))$, which is a tautology by the law of excluded middle.
3. **False.** As a countermodel, consider $(\{a, b\}, e)$ with $f^e(x) = x$ and $P^e(x, y) \iff x = y$.
4. **True.** We prove this by contradiction, transforming the negation of the formula into clausal form. We obtain:

- 1 : $\{\neg P(x, y), \neg P(y, z), P(x, z)\}$
- 2 : $\{P(x, y), P(y, x)\}$
- 3 : $\{\neg P(c, c)\}$

The resolution proof is as follows:

4 : $\{P(c, y), P(y, c)\}$ with $x \rightarrow c$ from clause 2

5 : $\{P(c, c)\}$ with $y \rightarrow c$ from clause 4

6 : $\{\}$ by resolving clauses 3 and 5 with $y \rightarrow c$

5. **False.** As a countermodel, consider $(\{a, b\}, e)$ with $f^e(x) = x$ and $P^e(x, y) \iff x = y$.

6. **True.** We prove this by contradiction, transforming the negation of the formula into clausal form. We obtain:

1 : $\{P(x, y)\}$

2 : $\{\neg P(a, b)\}$

The resolution proof is as follows:

3 : $\{P(x, b)\}$ with $y \rightarrow b$ from clause 1

4 : $\{P(a, b)\}$ with $x \rightarrow a$ from clause 3

5 : $\{\}$ by resolving clauses 2 and 4

◇

Exercise 2 (Weakest Precondition). Recall the definition of the weakest precondition operator:

$$\text{wp}(r, Q) = \{s \mid \forall s'. (s, s') \in r \rightarrow s' \in Q\}$$

1. Prove or disprove the following properties:

(a) $\text{wp}(r_1 \cup r_2, Q) = \text{wp}(r_1, Q) \cup \text{wp}(r_2, Q)$

(b) $\text{wp}(r_1 \cup r_2, Q) = \text{wp}(r_1, Q) \cap \text{wp}(r_2, Q)$

(c) $\text{wp}(r, Q_1 \cap Q_2) = \text{wp}(r, Q_1) \cap \text{wp}(r, Q_2)$

(d) $\text{wp}(r, Q_1 \cup Q_2) = \text{wp}(r, Q_1) \cup \text{wp}(r, Q_2)$

For those that are false, do any of them hold if r , r_1 , and r_2 are restricted to being a functional relation? A relation r is functional if it satisfies

$$\forall x, y_1, y_2. ((x, y_1) \in r \wedge (x, y_2) \in r) \rightarrow (y_1 = y_2) .$$

2. Let $r \subseteq S \times S$ and $Q \subseteq S$. Give an expression defining weakest precondition $\text{wp}(r, Q)$ using the operations inverse of a relation, $(\cdot^{-1})$, set difference $(\cdot \setminus \cdot)$, image of a set under a relation, $(\cdot[\cdot])$, and complement w.r.t S , $\cdot^c (= \cdot \setminus S)$. Prove that your expression is correct by expanding the definitions of wp as well as of the relational and set operations.

Solution:

1. (a) **False.**

Let $r_1 = \{(a, a), (b, a)\}$, $r_2 = \{(a, b), (b, b)\}$ and $Q = \{b\}$.

Then $\text{wp}(r_1, Q) = \emptyset$, $\text{wp}(r_2, Q) = \{a, b\}$ and $\text{wp}(r_1 \cup r_2, Q) = \emptyset$.

Note that r_1 and r_2 are functional relations already.

(b) **True.**

$$\begin{aligned}
 & \text{wp}(r_1 \cup r_2, Q) \\
 &= \{s \mid \forall s'. (s, s') \in r_1 \cup r_2 \rightarrow s' \in Q\} \\
 &= \{s \mid \forall s'. ((s, s') \in r_1 \vee (s, s') \in r_2) \rightarrow s' \in Q\} \\
 &= \{s \mid \forall s'. (s, s') \in r_1 \rightarrow s' \in Q \wedge (s, s') \in r_2 \rightarrow s' \in Q\} \\
 &= \{s \mid (\forall s'. (s, s') \in r_1 \rightarrow s' \in Q) \wedge (\forall s'. (s, s') \in r_2 \rightarrow s' \in Q)\} \\
 &= \{s \mid \forall s'. (s, s') \in r_1 \rightarrow s' \in Q\} \cap \{s \mid \forall s'. (s, s') \in r_2 \rightarrow s' \in Q\} \\
 &= \text{wp}(r_1, Q) \cap \text{wp}(r_2, Q)
 \end{aligned}$$

(c) **True.**

$$\begin{aligned}
 & \text{wp}(r, Q_1 \cap Q_2) \\
 &= \{s \mid \forall s'. (s, s') \in r \rightarrow s' \in Q_1 \cap Q_2\} \\
 &= \{s \mid \forall s'. (s, s') \in r \rightarrow (s' \in Q_1 \wedge s' \in Q_2)\} \\
 &= \{s \mid \forall s'. ((s, s') \in r \rightarrow s' \in Q_1) \wedge ((s, s') \in r \rightarrow s' \in Q_2)\} \\
 &= \{s \mid (\forall s'. (s, s') \in r \rightarrow s' \in Q_1) \wedge (\forall s'. (s, s') \in r \rightarrow s' \in Q_2)\} \\
 &= \{s \mid \forall s'. (s, s') \in r \rightarrow s' \in Q_1\} \cap \{s \mid \forall s'. (s, s') \in r \rightarrow s' \in Q_2\} \\
 &= \text{wp}(r, Q_1) \cap \text{wp}(r, Q_2)
 \end{aligned}$$

(d) **False** in general, but **true** when r is functional:

Let $r = \{(a, b), (a, c)\}$, $Q_1 = \{b\}$, $Q_2 = \{c\}$.

We have $\text{wp}(r, Q_1) = \text{wp}(r, Q_2) = \emptyset$ but $\text{wp}(r, Q_1 \cup Q_2) = \{a\}$.

When r is a partial function (i.e. a functional relation) the weakest precondition becomes the union between the preimage and the set of all the elements that are not in the domain. This is due to the fact that when r is a partial function the image of an element is either a singleton or the empty set. Let s be an arbitrary element, if s is not in the domain then

$$\forall s'. (s, s') \in r \rightarrow s' \in Q$$

is vacuously true for each s' . If s is in the domain then the statement reduces to whether its image is in Q , which is equivalent to $\exists s'. (s, s') \in r \wedge s' \in Q$. The set of elements verifying this property is the preimage of Q under r also noted $r^{-1}[Q]$. Therefore, we have $\text{wp}(r, Q) = r^{-1}[Q] \cup \{s \mid \forall s'. (s, s') \notin r\}$.

The required statement follows from this observation. We first show that the preimage of a union is the union of the preimages.

$$\begin{aligned}
r^{-1}[Q_1 \cup Q_2] &= \{s \mid \exists s'. (s, s') \in r \wedge s' \in Q_1 \cup Q_2\} \\
&= \{s \mid \exists s'. (s, s') \in r \wedge (s' \in Q_1 \vee s' \in Q_2)\} \\
&= \{s \mid \exists s'. ((s, s') \in r \wedge s' \in Q_1) \vee ((s, s') \in r \wedge s' \in Q_2)\} \\
&= \{s \mid (\exists s'. (s, s') \in r \wedge s' \in Q_1) \vee (\exists s'. (s, s') \in r \wedge s' \in Q_2)\} \\
&= \{s \mid \exists s'. (s, s') \in r \wedge s' \in Q_1\} \cup \{s \mid \exists s'. (s, s') \in r \wedge s' \in Q_2\} \\
&= r^{-1}[Q_1] \cup r^{-1}[Q_2]
\end{aligned}$$

We thus have:

$$\begin{aligned}
\text{wp}(r, Q_1 \cup Q_2) &= r^{-1}[Q_1 \cup Q_2] \cup \{s \mid \forall s'. (s, s') \notin r\} \\
&= r^{-1}[Q_1] \cup r^{-1}[Q_2] \cup \{s \mid \forall s'. (s, s') \notin r\} \\
&= r^{-1}[Q_1] \cup \{s \mid \forall s'. (s, s') \notin r\} \\
&\quad \cup r^{-1}[Q_2] \cup \{s \mid \forall s'. (s, s') \notin r\} \\
&= \text{wp}(r, Q_1) \cup \text{wp}(r, Q_2)
\end{aligned}$$

The last two steps use the fact that the set of elements which are not in the domain do not depend on Q_1 or Q_2 .

2. We can extend the observation made in 1d in the case of functional relations to arbitrary relations. We have for a functional relation r :

$$\text{wp}(r, Q) = r^{-1}[Q] \cup \{s \mid \forall s'. (s, s') \notin r\}.$$

In the case of a non-functional relation, the elements in the second relation are indeed contained in $\text{wp}(r, Q)$. However, there are elements s in $r^{-1}[Q]$ such that $\exists s' (s, s') \in r \wedge s' \notin Q$, i.e. elements which map to some elements in Q , but also to some elements not in Q . We can thus remove these elements explicitly, to get:

$$\text{wp}(r, Q) \stackrel{?}{=} (r^{-1}[Q] \setminus r^{-1}[Q^c]) \cup \{s \mid \forall s'. (s, s') \notin r\}.$$

where $Q^c = S \setminus Q$. This is the same as

$$\text{wp}(r, Q) \stackrel{?}{=} r^{-1}[Q^c]^c,$$

or all states which do not map to states not in Q . We can prove that this equality holds by expanding the definitions.

$$\begin{aligned}
& r^{-1}[Q^c]^c \\
&= \{s \in S \mid \exists s'. (s, s') \in r \wedge s' \in Q^c\}^c \\
&= \{s \in S \mid \exists s'. (s, s') \in r \wedge s' \in (S \setminus Q)\}^c \\
&= \{s \in S \mid \exists s'. (s, s') \in r \wedge (s' \in S \wedge s' \notin Q)\}^c \\
&= S \setminus \{s \in S \mid \exists s'. (s, s') \in r \wedge (s' \in S \wedge s' \notin Q)\} \\
&= \{s \in S \mid \neg \exists s'. (s, s') \in r \wedge (s' \in S \wedge s' \notin Q)\} \quad (\dagger) \\
&= \{s \in S \mid \forall s'. \neg((s, s') \in r \wedge (s' \in S \wedge s' \notin Q))\} \\
&= \{s \in S \mid \forall s'. (s, s') \notin r \vee s' \notin S \vee s' \in Q\} \\
&= \{s \in S \mid \forall s'. ((s, s') \in r \wedge s' \in S) \rightarrow s' \in Q\} \\
&= \{s \in S \mid \forall s'. (s, s') \in r \rightarrow s' \in Q\} \quad (*) \\
&= wp(r, Q)
\end{aligned}$$

where step $(\dagger)$ is a fact about set comprehensions, and step $(*)$ follows as $r \subseteq S \times S$, so $(s, s') \in r$ implies $s' \in S$.

◇

Exercise 3 (Programming With Integers). Consider the following program:

```

1 import stainless.lang.*
2 type INT = BigInt
3 case class Container(var x: INT, var y: INT):
4   def fun: Unit = {
5     require(x > 0 && y > 0)
6     if x > y then
7       x = x + y
8       y = x - y
9       x = x - y
10    else
11      y = y - x
12  }.ensuring(2 * old(this).x + old(this).y > 2 * x + y &&
13            x >= 0 && y >= 0)

```

Take I as the set representing values of type `INT`.

1. Compute $R(\text{fun}) \subseteq I^4$, the relation corresponding to the imperative program represented by `fun`, relating the previous state (x, y) to the final state (x', y') , by expressing all intermediate formulas corresponding to subprograms.
2. Write the formula expressing the correctness of the ensuring clause.

3. Is the verification condition formula valid when INT denotes mathematical integers with their usual operations (**type** INT = BigInt in Scala), i.e. when $I = \mathbb{Z}$?
4. Is the verification condition formula valid when INT denotes 32-bit machine integers and where operations are interpreted as the usual machine operations (**type** INT = Int in Scala), i.e. when $I = \mathbb{Z}_{2^{32}}$?

$$\mathbb{Z}_{2^{32}} = \{-2^{31}, -2^{31} + 1, \dots, -1, 0, 1, \dots, 2^{31} - 1\}$$

Solution:

1. Using the following translation from composite commands to formulas:

Command	Formula
$f; s$	$\exists x_i, y_i. R(f) [x', y' := x_i, y_i] \wedge R(s) [x, y := x_i, y_i]$
$\text{if } c \text{ then } t \text{ else } e$	$(R(c) \wedge R(t)) \vee (\neg R(c) \wedge R(e))$

We get:

Command	Formula
$x = x+y$	$F_1 := x' = x + y \wedge y' = y$
$y = x-y$	$F_2 := x' = x \wedge y' = x - y$
$x = x-y$	$F_3 := x' = x - y \wedge y' = y$
$y = y-x$	$F_4 := x' = x \wedge y' = y - x$
if body	$F_{\text{if}} := \exists x_{i1}, y_{i1}, x_{i2}, y_{i2}. \\ F_1 [x', y' := x_{i1}, y_{i1}] \wedge \\ F_2 [x, y, x', y' := x_{i1}, y_{i1}, x_{i2}, y_{i2}] \wedge \\ F_3 [x, y := x_{i2}, y_{i2}]$
else body	$F_{\text{else}} := F_4$
program	$F := ((x > y) \wedge F_{\text{if}}) \vee (\neg(x > y) \wedge F_{\text{else}})$

2. We first express the pre- and post-condition:

$$P(x, y) = x > 0 \wedge y > 0$$

$$Q(x, y, x', y') = 2x + y > 2x' + y' \wedge x' > 0 \wedge y' > 0$$

We can then express the formula asserting the correctness of **fun**:

$$V = \forall x, y, x', y'. (P(x, y) \wedge F(x, y, x', y')) \rightarrow Q(x, y, x', y')$$

3. If $I = \mathbb{Z}$, then we can note that this function amounts to:

- If $x > y$, swap x and y ;
- Else, decrease y by x .

Case analysis on the branches allows us to prove the validity of the formula V above.

4. We note $\oplus$ for wrapping addition (e.g. $(2^{31} - 1) \oplus 1 = -2^{31}$).

The body of the program still behaves as suggested. However, the postcondition doesn't hold: consider $x = 1$, $y = 2^{31} - 1$. Then $x' = 1$, $y' = 2^{31} - 2$. The first postcondition is

$$2^{31} \oplus 1 = 2 \oplus (2^{31} - 1) = 2x \oplus y \stackrel{?}{>} 2x' \oplus y' = 2 \oplus (2^{31} - 2) = 2^{31}$$

However, due to wrapping, $2^{31} \oplus 1 = -2^{31} < 0$.

If you want to test this with `stainless`, you have to use the option `--strict-arithmetic=false`.

◇

Exercise 4 (Hoare Logic Proof). Give a complete Hoare logic proof for the following program:

```
{n >= 0 && d > 0}
  q = 0
  r = n
  while ( r >= d ) {
    q = q + 1
    r = r - d
  }
{n == q * d + r && 0 <= r < d}
```

The proof should include step-by-step annotation for each line of the program, as in the example proof in the lecture.

Solution:

```
// {n >= 0 && d > 0}
q = 0
// {n >= 0 && d > 0 && q == 0}
r = n
// {n >= 0 && d > 0 && q == 0 && r == n}
while // {d > 0 && n == q * d + r && 0 <= r}
  (r >= d) {
    // {d > 0 && n == q * d + r && d <= r}
    q = q + 1
    // {d > 0 && n == (q-1) * d + r && d <= r}
    r = r - d
    // {d > 0 && n == (q-1) * d + r + d && 0 <= r}
    // {d > 0 && n == q * d + r && 0 <= r}
```

```

}
// {d > 0 && n == q * d + r && 0 <= r && r < d}
// {n == q * d + r && 0 <= r < d}

```

◇

Exercise 5 (Iterating a Relation). Let $M = (S, I, r, A)$ be a transition system and $\bar{r} = \{(s, s') \mid (s, a, s') \in r\}$, as usual. Let $\Delta = \{(x, x) \mid x \in S\}$.

Let $\bar{r}^k$ denote the usual composition of relation $\bar{r}$ with itself k times.

Define the sequence of relations r_n , for all non-negative integers n , as follows:

- $r_0 = \Delta \cup \bar{r}$
- $r_{n+1} = r_n \circ r_n$

1. Prove that $r_n \subseteq r_{n+1}$ for every n .

Solution: *By induction on n :*

Base case: ($n = 0$)

$$\begin{aligned}
r_1 &= r_0 \circ r_0 \\
&= (\Delta \cup \bar{r}) \circ (\Delta \cup \bar{r}) \\
&= (\Delta \circ \Delta) \cup (\Delta \circ \bar{r}) \cup (\bar{r} \circ \Delta) \cup (\bar{r} \circ \bar{r}) \\
&= \Delta \cup \bar{r} \cup \bar{r}^2 \\
&= r_0 \cup \bar{r}^2
\end{aligned}$$

In the steps above, we used the fact that composition distributes over union (c.f. Exercise Set 2 part 2), and that Δ is a neutral element w.r.t. composition (since $\Delta = \bar{r}^0$).

Clearly, $r_0 \subseteq r_0 \cup \bar{r}^2 = r_1$.

Induction: (induction hypothesis: $r_{n-1} \subseteq r_n$)

We have:

$$\begin{aligned}
r_n &= r_{n-1} \circ r_{n-1} \\
&\subseteq r_n \circ r_n & (*) \\
&= r_{n+1}
\end{aligned}$$

where step () follows from the induction hypothesis and the fact that if $r_1 \subseteq r'_1$ and $r_2 \subseteq r'_2$ then $r_1 \circ r_2 \subseteq r'_1 \circ r'_2$*

◇

2. Prove that for every n and every k where $0 \leq k \leq 2^n$ we have $\bar{r}^k \subseteq r_n$.

Solution: By induction on n :

Base case: ($n = 0$)

$$\bar{r}^0 = \Delta \subseteq \Delta \cup \bar{r} = r_0 \quad \bar{r}^1 = \bar{r} \subseteq \Delta \cup \bar{r} = r_0$$

Induction: (induction hypothesis: $\forall k. 0 \leq k \leq 2^n$ we have $\bar{r}^k \subseteq r_n$)

We split cases based on whether $k \leq 2^n$ or not:

- (a) $0 \leq k \leq 2^n$: then $\bar{r}^k \subseteq r_n$ by IH. From the previous part, $r_n \subseteq r_{n+1}$, so $\bar{r}^k \subseteq r_{n+1}$.
- (b) $2^n < k \leq 2^{n+1}$: then k can be split into two parts, each less than or equal to 2^n :

$$\begin{aligned} \bar{r}^k &= \bar{r}^{2^n} \circ \bar{r}^{k-2^n} \\ &\subseteq r_n \circ r_n \\ &= r_{n+1} \end{aligned}$$

which follows by the induction hypothesis and monotonicity of composition as before.

◇

3. Suppose that S is finite. Find a bound B as a function of $|S|$ such that

$$\text{Reach}(M) \subseteq r_B[I]$$

Aim to find as small bound as possible.

Solution: Since there are $|S|$ states, every state must be reachable in at most $|S|$ steps,

$$\text{Reach}(M) \subseteq \bigcup_{k=0}^{|S|} \bar{r}^k[I] \subseteq r_{\lceil \log_2 |S| \rceil}[I]$$

In fact, $\lceil \log_2 |S| \rceil$ is the smallest n such that $\bar{r}^k[I] \subseteq r_n[I]$ for every k such that $0 \leq k \leq 2^n$. ◇

Exercise 6 (Approximating Relations). Consider a guarded command language operating on a state. The set of all states is denoted by U . The language contains the following commands, constructed inductively:

- atomic commands $\{a_1, a_2, \dots\}$, representing arbitrary user-defined state transitions

- non-deterministic choice: **if** (*) c_1 **else** c_2
- sequential composition: $c_1; c_2$
- assume statement: **assume** G , where the guard G is a predicate on states

where c, c_1, c_2 denote commands.

We interpret programs in this language as relations on states ($\subseteq U \times U$), relating the input state to possible output states.

Corresponding to each program, we build an expression denoting its characteristic relation. The language of relations is constructed inductively:

- atomic relations $r_1, r_2, \dots \subseteq U \times U$
- diagonal relations $\Delta_P = \{(x, x) \mid x \in P\}$ for $P \subseteq U$
- union of relations ($\cdot \cup \cdot$)
- composition of relations ($\cdot \circ \cdot$)

We can define the translation from commands to relational expressions (ρ) as follows:

c	$\rho(c)$
a_i	r_i
if (*) c_1 else c_2	$\rho(c_1) \cup \rho(c_2)$
$c_1; c_2$	$\rho(c_1) \circ \rho(c_2)$
assume G	$\Delta_{S(G)}$

where $\Delta_{S(G)}$ denotes the diagonal relation for the set of states satisfying G , i.e. Δ_P where $P = \{s \in U \mid G(s)\}$.

The atomic relations r_i can be constructed for each atomic command. If the state consists of two integer variables, i.e., if $U = \mathbb{Z} \times \mathbb{Z}$, then the atomic command $\mathbf{x} := \mathbf{y} + 1$, for example, can be represented by the relation

$$r((x, y), (x', y')) \iff x' = y + 1 \wedge y' = y.$$

By $E(a_1, \dots, a_n)$ we denote an arbitrary relational expression built from some atomic relations $a_1, \dots, a_n$.

Let us call a relation $e \subseteq U \times U$ an *effect* if it is reflexive and transitive. We say that an effect e *approximates* an expression $E(a_1, \dots, a_n)$ if $E(a_1, \dots, a_n) \subseteq e$.

1. Assuming $U = \mathbb{Z} \times \mathbb{Z}$, for the following atomic commands, compute the atomic relations corresponding to them:

(a) $\mathbf{x} := \mathbf{x} + 1$

- (b) $y := y + x$
- (c) $x := x$
- (d) **havoc**(x) (non-deterministically assign x)

Solution:

- (a) $r_a((x, y), (x', y')) \iff x' = x + 1 \wedge y' = y,$
- (b) $r_b((x, y), (x', y')) \iff x' = x \wedge y' = y + x,$
- (c) $r_c((x, y), (x', y')) \iff x' = x \wedge y' = y, \text{ and}$
- (d) $r_d((x, y), (x', y')) \iff y' = y.$

◇

2. Compute the relation $\rho(C)$ for the following program C :

$y := x + y ; x := y - x ; y := y - x$

Solution: *For the atomic commands, we have:*

$$\begin{aligned} \rho(y := x + y) &= \{((x, y), (x', y')) \mid x' = x \wedge y' = x + y\} \\ \rho(x := y - x) &= \{((x, y), (x', y')) \mid x' = y - x \wedge y' = y\} \\ \rho(y := y - x) &= \{((x, y), (x', y')) \mid x' = x \wedge y' = y - x\} \end{aligned}$$

For C , evaluating based on rules for relation composition and integer arithmetic, we have:

$$\begin{aligned} \rho(C) &= \rho(y := x + y) \circ \rho(x := y - x) \circ \rho(y := y - x) \\ &= \{((x, y), (x', y')) \mid x' = x + y - x \wedge y' = x + y\} \circ \rho(y := y - x) \\ &= \{((x, y), (x', y')) \mid x' = y \wedge y' = x + y\} \circ \rho(y := y - x) \\ &= \{((x, y), (x', y')) \mid x' = y \wedge y' = x + y - y\} \\ &= \{((x, y), (x', y')) \mid x' = y \wedge y' = x\} \end{aligned}$$

◇

3. Prove that if a relation e is an effect and $r_i \subseteq e$ for all $1 \leq i \leq n$, then

$$\forall E. E(a_1, \dots, a_n) \subseteq e$$

which is to say that if e approximates a set of atomic relations, then it approximates any expression built from those relations.

Solution: *We prove this by structural induction on the relational expression E :*

Base case: ($E = a_i$ for some $1 \leq i \leq n$, or $E = \Delta_P$ for some $P \subseteq U$)

$a_i \subseteq e$ for all $1 \leq i \leq n$ by assumption, and since e is reflexive, $\Delta_P \subseteq e$ for every $P \subseteq U$.

Induction:

We have three cases to consider:

$$(a) \ E(r_1, \dots, r_n) = E_1(r_1, \dots, r_n) \cup E_2(r_1, \dots, r_n)$$

$$(b) \ E(r_1, \dots, r_n) = E_1(r_1, \dots, r_n) \circ E_2(r_1, \dots, r_n)$$

$$(c) \ E(r_1, \dots, r_n) = E_1(r_1, \dots, r_n)^*$$

with the induction hypothesis that $E_1(r_1, \dots, r_n), E_2(r_1, \dots, r_n) \subseteq e$.

$$(a) \ E(r_1, \dots, r_n) = E_1(r_1, \dots, r_n) \cup E_2(r_1, \dots, r_n)$$

$$\begin{aligned} E(a_1, \dots, a_n) &= E_1(a_1, \dots, a_n) \cup E_2(a_1, \dots, a_n) \\ &\subseteq e \cup e \\ &= e \end{aligned}$$

where the inclusion is due to IH.

$$(b) \ E(r_1, \dots, r_n) = E_1(r_1, \dots, r_n) \circ E_2(r_1, \dots, r_n)$$

$$\begin{aligned} E(a_1, \dots, a_n) &= E_1(a_1, \dots, a_n) \circ E_2(a_1, \dots, a_n) \\ &\subseteq e \circ e \\ &= e \end{aligned}$$

where inclusion is due to monotonicity of composition and the IH, and the last equality to the fact that e is transitive.

$$(c) \ E(r_1, \dots, r_n) = E_1(r_1, \dots, r_n)^*$$

$$\begin{aligned} E(a_1, \dots, a_n) &= E_1(a_1, \dots, a_n)^* \\ &\subseteq e^* \\ &= e \end{aligned}$$

where inclusion is due to IH and the last equality to the fact that e is both transitive and reflexive.

◇

4. For each of the following representations of relations, prove or disprove whether they are effects. Assume as before that $U = \mathbb{Z} \times \mathbb{Z}$, with a state having two variables x and y . Further, informally, what can we say about programs whose relations are approximated by these effects?

- (a) $\rho(\text{havoc}(x))$
- (b) Δ_U
- (c) $\{((x, y), (x', y')) \mid y \geq 0 \rightarrow (x' \leq x \wedge y' \geq 0)\}$

Solution:

- (a) $e_a = \rho(\text{havoc}(x)) = \{((x, y), (x', y')) \mid y' = y\}$.

- i. *Reflexivity:* for arbitrary (x, y) , clearly $((x, y), (x, y)) \in e_a$ since $y = y$.
- ii. *Transitivity:* let $((x_1, y_1), (x_2, y_2)), ((x_2, y_2), (x_3, y_3)) \in e_a$. Then $y_1 = y_2$ and $y_2 = y_3$, so $y_1 = y_3$, and therefore $((x_1, y_1), (x_3, y_3)) \in e_a$.

This effect approximates programs that change x but not y .

- (b) $e_b = \Delta_U = \{((x, y), (x', y')) \mid x = x' \wedge y = y'\}$. This is a diagonal relation, and is reflexive and transitive by definition.

This effect approximates programs that do not change the state.

- (c) $e_c = \{((x, y), (x', y')) \mid y \geq 0 \rightarrow (x' \leq x \wedge y' \geq 0)\}$.

- i. *Reflexivity:* for arbitrary (x, y) , if $y < 0$, then the condition is vacuously true; if $y \geq 0$, then $x' = x$ and $y' = y \geq 0$ satisfy the condition. Therefore, $((x, y), (x, y)) \in e_c$.
- ii. *Transitivity:* let $((x_1, y_1), (x_2, y_2)), ((x_2, y_2), (x_3, y_3)) \in e_c$. The following conditions hold:

$$\begin{aligned} y_1 \geq 0 &\rightarrow (x_2 \leq x_1 \wedge y_2 \geq 0) \\ y_2 \geq 0 &\rightarrow (x_3 \leq x_2 \wedge y_3 \geq 0) \end{aligned}$$

We need to show that $y_1 \geq 0 \rightarrow (x_3 \leq x_1 \wedge y_3 \geq 0)$.

If $y_1 < 0$, then the condition is vacuously true. If $y_1 \geq 0$, then by the first condition, $x_2 \leq x_1$ and $y_2 \geq 0$. Since $y_2 \geq 0$, by the second condition, $x_3 \leq x_2$ and $y_3 \geq 0$. Therefore, $x_3 \leq x_1$ and $y_3 \geq 0$.

This effect approximates programs that do not increase x and do not change y 's parity when y is non-negative to begin with.

◇

5. For each of the following programs, argue whether they are approximated by each of the effects you identified in the previous problem:

- (a) `x := x + y ; y = x - y ; x := x - y`
- (b) `if (*) x := x - 2 else x := x - 1`
- (c) `assume y > 2 ; x := x - 1 ; y := y - 2`
- (d) `assume y > 0`

Solution:

We have three effects from the previous part:

- $e_a = \rho(\text{havoc}(x)) = \{((x, y), (x', y')) \mid y' = y\}$
- $e_b = \Delta_U = \{((x, y), (x', y')) \mid x = x' \wedge y = y'\}$
- $e_c = \{((x, y), (x', y')) \mid y \geq 0 \rightarrow (x' \leq x \wedge y' \geq 0)\}$

We consider each program in turn:

- (a) `x := x + y ; y = x - y ; x := x - y`
 - e_a : ✗. The pair $((0, 1), (1, 0))$ is in the program's relation but not in e_a .
 - e_b : ✗. Same counterexample as above.
 - e_c : ✗. Same counterexample as above.
- (b) `if (*) x := x - 2 else x := x - 1`
 - e_a : ✓. Each atomic command is in e_a , so by part 3, the whole program is in e_a .
 - e_b : ✗. Consider the pair $((0, 1), (-1, 1))$, which is in the program's relation but not in e_b .
 - e_c : ✓. Each atomic command is in e_c , so by part 3, the whole program is in e_c .
- (c) `assume y > 2 ; x := x - 1 ; y := y - 2`
 - e_a : ✗. The pair $((1, 2), (0, 0))$ is in the program's relation but not in e_a .
 - e_b : ✗. Same counterexample as above.

- e_c : ✓. Note that the atomic command $y := y - 2$ is not in e_c . However, the program relation is:

$$\{((x, y), (x', y')) \mid y > 2 \wedge x' = x - 1 \wedge y' = y - 2\},$$

which is indeed a subset of e_c . It follows that the conditions of part 3 are sufficient but not necessary for approximation.

In particular, it does not take context into account.

(d) **assume** $y > 0$

Note that the set of atomic instructions is empty in this case, so it is approximated by every effect vacuously by part 3.

- e_a : ✓.
- e_b : ✓.
- e_c : ✓.

◇